

3-Tier Microservices DevSecOps Platform

End-to-end CI/CD · GitOps Delivery · Security-First · Full Observability

Architecture • Pipeline • Observability • Challenges • Quick Start

Overview

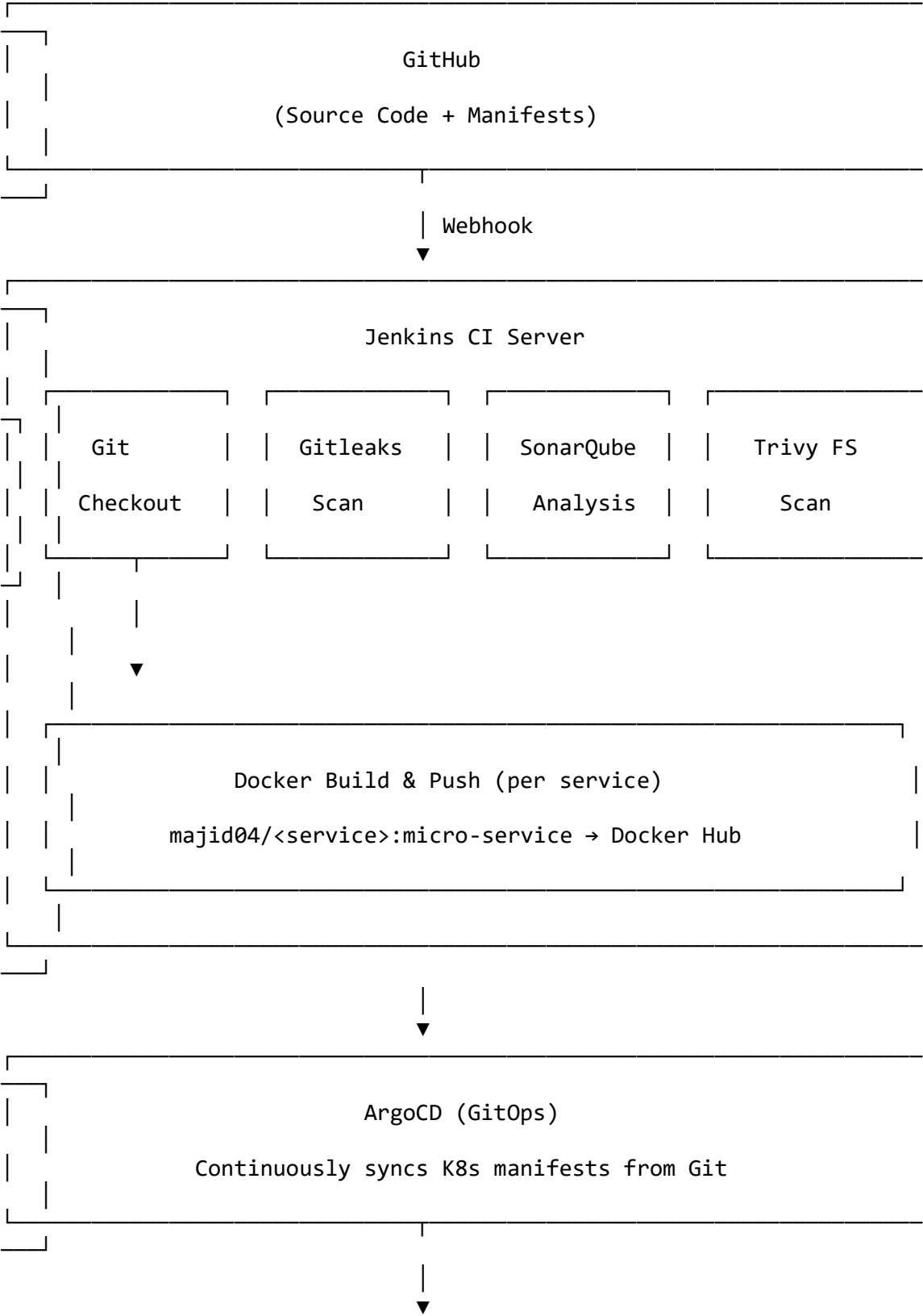
This project demonstrates a **production-grade DevSecOps platform** built around Google's [microservices-demo](#) application. The goal was not just to deploy an app, but to engineer a complete **secure delivery pipeline** with automated testing, vulnerability scanning, GitOps synchronization, and real-time observability — all running on a lightweight K3s cluster.

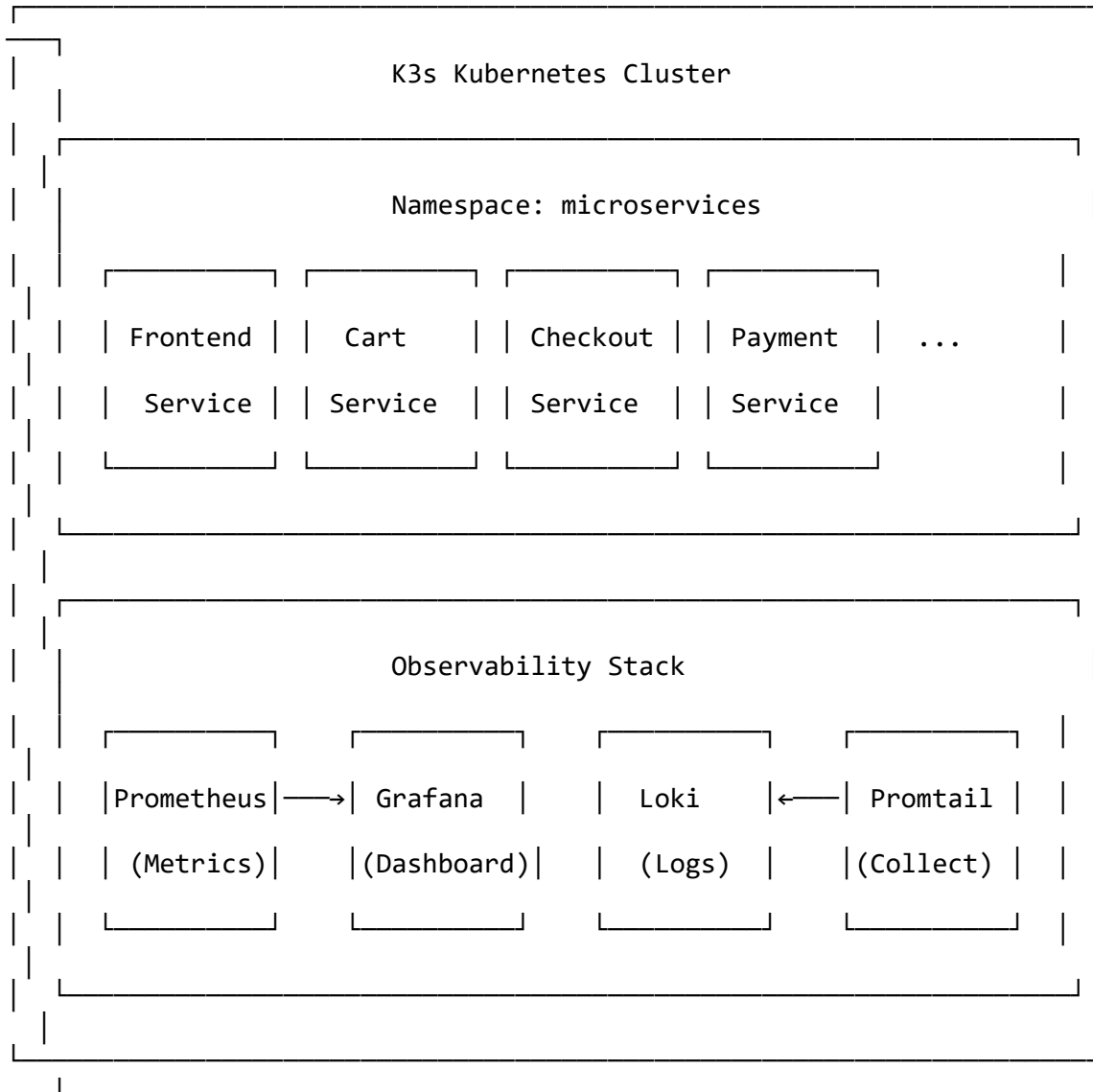
Built in 1 day. Production-ready mindset.

What Makes This Project Stand Out

Capability	Implementation
Security-First CI	Every code push triggers secret scanning (Gitleaks), SAST (SonarQube), and dependency scanning (Trivy) before a single image is built
GitOps Delivery	ArgoCD continuously reconciles cluster state against Git, eliminating configuration drift
Zero-Downtime Deployments	Rolling updates via Kubernetes with health checks and readiness probes
Full Observability	Prometheus metrics → Grafana dashboards + Loki logs → unified troubleshooting
Namespace Isolation	All microservices isolated in a dedicated <code>microservices</code> namespace
Automated Image Distribution	Docker images built, tagged, and pushed to Docker Hub automatically

Architecture





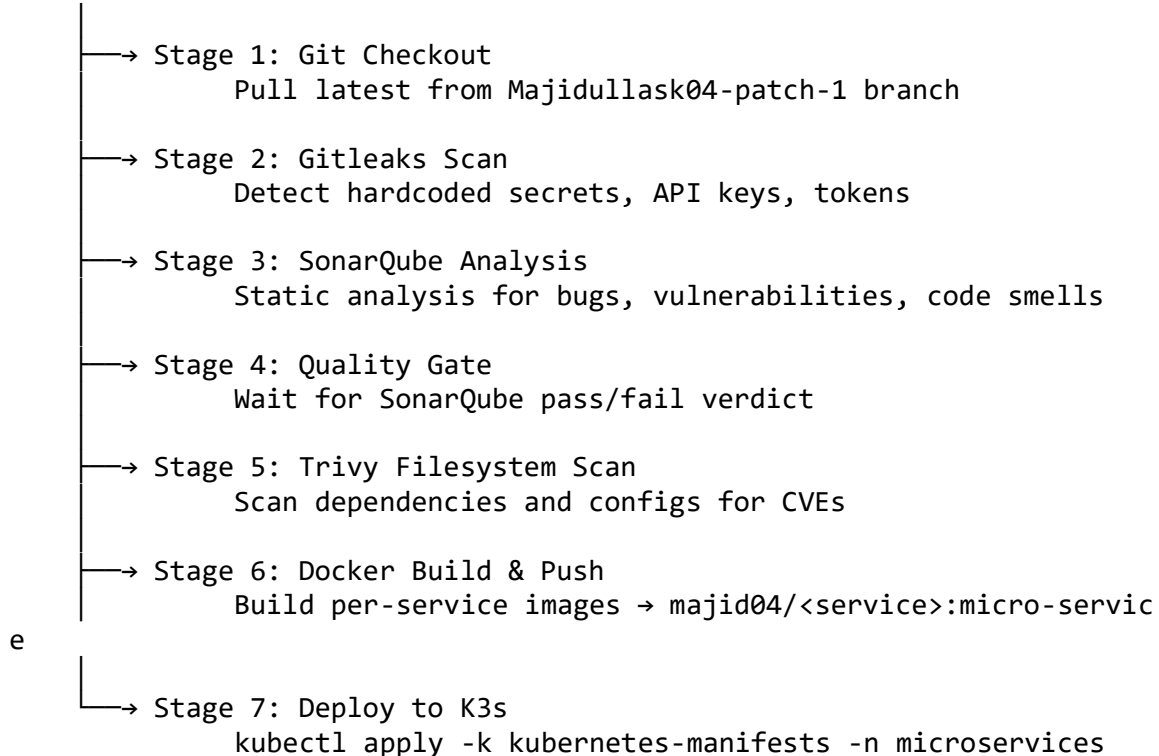
Tech Stack

Layer	Tool	Purpose
Virtualization	OrbStack	Lightweight macOS virtualization
OS	Ubuntu VM	Development and cluster host
Container Runtime	Docker	Image building and local runtime
Orchestration	K3s	Lightweight Kubernetes distribution
CI/CD	Jenkins	Declarative pipeline automation

Layer	Tool	Purpose
GitOps	ArgoCD	Declarative continuous delivery
Secret Scanning	Gitleaks	Prevent credential leakage
SAST	SonarQube	Code quality and security analysis
Vulnerability Scan	Trivy	Container and filesystem scanning
Image Registry	Docker Hub	Centralized image distribution
Metrics	Prometheus	Time-series metrics collection
Visualization	Grafana	Dashboards and alerting UI
Log Aggregation	Loki	Centralized log storage
Log Collection	Promtail	Pod log forwarding to Loki
Deployment	Kustomize	Kubernetes manifest templating

CI/CD Pipeline

Git Push



Pipeline Screenshots

Place your Jenkins pipeline screenshot here Jenkins Pipeline

GitOps with ArgoCD

ArgoCD watches the Git repository and automatically synchronizes the desired state with the live K3s cluster.

Feature	Status
Auto-Sync	✓ Enabled
Drift Detection	✓ Active
Resource Tree Visualization	✓ Verified
Rollback Capability	✓ Tested

Place your ArgoCD Application Tree screenshot here ArgoCD Tree

Observability

Metrics (Prometheus + Grafana)

- **Node Resource Overview:** CPU, Memory, Disk utilization
- **Namespace Resource Tracking:** Per-namespace workload counts
- **Network Overview:** Receive/Send throughput across pods
- **Pod Health:** Real-time status of all microservices

Key Metrics at a Glance: - Workloads: 25 - Total Pods: 31 - Total Nodes: 1 - Memory Usage: ~2 GiB active / 8 GiB total - CPU Cores: 8.0 total, 0.4 requested

Place your Grafana dashboard screenshot here Grafana Dashboard

Log Aggregation (Loki + Promtail)

All pod logs are collected by Promtail and centralized in Loki, queryable directly from Grafana.

Example Log query in Grafana

```
{namespace="microservices", app="frontend"} |= "error"
```

Place your Loki log panel screenshot here Loki Logs

Challenges & Solutions

1. ImagePullBackOff on Deployment

	Details
Symptom	Pods stuck in ImagePullBackOff status
Root Cause	Kubernetes could not resolve image references in deployment manifests
Solution	Updated all image names to match Docker Hub repository format: majid04/<service>:micro-service
Lesson	Always verify image pull secrets and repository naming conventions before deployment

2. Cart Service Build Failure

	Details
Symptom	Jenkins pipeline failed only on cartservice build step
Root Cause	Cart Service Dockerfile was nested at src/cartservice/src/Dockerfile instead of src/cartservice/Dockerfile like other services
Solution	Modified Jenkins build logic to dynamically extract service names from Dockerfile paths using <code>cut -d'/' -f2</code>

	Details
Lesson	Heterogeneous repository structures require adaptive CI scripts, not hardcoded assumptions
3. K3s API Server Connectivity Failure	
	Details
Symptom	kubectl commands returned connection refused 127.0.0.1:6443
Root Cause	K3s API server was temporarily unavailable
Solution	Restarted K3s services and verified kubeconfig context
Lesson	Local K3s clusters require service health checks before CI deployment stages
4. Kustomize Deployment Error	
	Details
Symptom	kubectl apply -f failed with no matches for kind "Kustomization"
Root Cause	Used plain kubectl apply -f instead of Kustomize-aware command
Solution	Switched to kubectl apply -k kubernetes-manifests
Lesson	Understand your manifest structure — Kustomize directories require -k, not -f

Project Structure

```
microservices-devops-platform/
├── README.md                    # This documentation
├── Jenkinsfile                  # Declarative CI/CD pipeline
├── kubernetes-manifests/       # Kustomize deployment configs
│   ├── kustomization.yaml
│   ├── frontend.yaml
│   ├── cartservice.yaml
│   ├── checkoutservice.yaml
│   ├── paymentsservice.yaml
│   └── ...
├── docs/
│   └── screenshots/           # Proof-of-work screenshots
│       ├── jenkins-pipeline.png
│       ├── argocd-tree.png
│       ├── grafana-dashboard.png
│       ├── prometheus-targets.png
│       └── kubectl-pods.png
└── scripts/
    └── build-and-push.sh      # Helper script for image builds
```

Quick Start

Prerequisites

- OrbStack (or Docker Desktop) with Ubuntu VM
- K3s cluster running locally
- Jenkins server with NodeJS 20, Sonar Scanner, and Docker configured
- ArgoCD installed on K3s
- Docker Hub account with credentials configured in Jenkins

Deploy the Application

1. Create namespace

```
kubectl create namespace microservices
```

2. Deploy via Kustomize

```
kubectl apply -n microservices -k kubernetes-manifests/
```

3. Verify pods

```
kubectl get pods -n microservices
```

4. Check services

```
kubectl get svc -n microservices
```


Access the Stack

Service	URL
Frontend App	http://<node-ip>:<frontend-port>
Grafana	http://localhost:3000
ArgoCD	http://localhost:8080
Prometheus	http://localhost:9090
Jenkins	http://localhost:8080 (Jenkins instance)

Skills Demonstrated

- **CI/CD Engineering:** Designed and implemented a multi-stage Jenkins pipeline with security gates
 - **Containerization:** Built, tagged, and distributed multi-service Docker images
 - **Kubernetes Operations:** Deployed and troubleshot workloads on K3s with namespace isolation
 - **GitOps Practices:** Configured ArgoCD for declarative, automated cluster synchronization
 - **Security Integration:** Embedded SAST, secret scanning, and vulnerability scanning into the delivery pipeline
 - **Observability Engineering:** Deployed Prometheus/Grafana/Loki stack for metrics and log aggregation
 - **Production Troubleshooting:** Systematically diagnosed and resolved ImagePullBackOff, API connectivity, and build path issues
 - **Documentation:** Produced technical documentation and architecture diagrams for stakeholder communication
-

Future Roadmap

Enhancement	Tool	Priority
Infrastructure as Code	Terraform	High
Helm Chart Packaging	Helm	High
Cloud Migration	AWS EKS / GKE	Medium
Runtime Security Policies	Falco	Medium
Distributed Tracing	Jaeger + OpenTelemetry	Medium
Secret Management	External Secrets Operator	Medium
Alerting	AlertManager + PagerDuty	Low
Multi-Environment GitOps	Staging + Production ArgoCD Apps	Low

Proof of Work

 **Video Walkthrough:** [Loom Link Placeholder] — 2-minute demo of the full pipeline

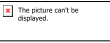
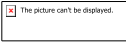
 **Screenshots:** See docs/screenshots/ directory

 **Application Source:** [Google Microservices Demo](#)

Author

Majidulla SK

DevOps Engineer | Cloud & Kubernetes Enthusiast | Open Source Learner

  <https://www.linkedin.com/in/majidulla-sk-1190a2286> 

<https://github.com/Majidullask04>  <mailto:majidullask04@gmail.com>

Built with persistence, debugged with patience, deployed with confidence.